



如何基于 Pi 开发自己的 Agent 项目

00 序篇 | Pi Harness 怎样让 Agent 跑起来

大家好，我是居丽叶。

从这篇开始，我会连载一个新系列——《如何基于 Pi 开发自己的 Agent 项目》。总共五篇：

- 00 序篇：Pi Harness 怎样让 Agent 跑起来
- 01 看懂 Pi：选对开发层，跑通第一个 Agent
- 02 组装 Pi：Tool、Context 与输出合约
- 03 组织 Pi：Router、Workers 与 Verifier
- 04 交付 Pi：从一次真实请求走到 Trace 与降级

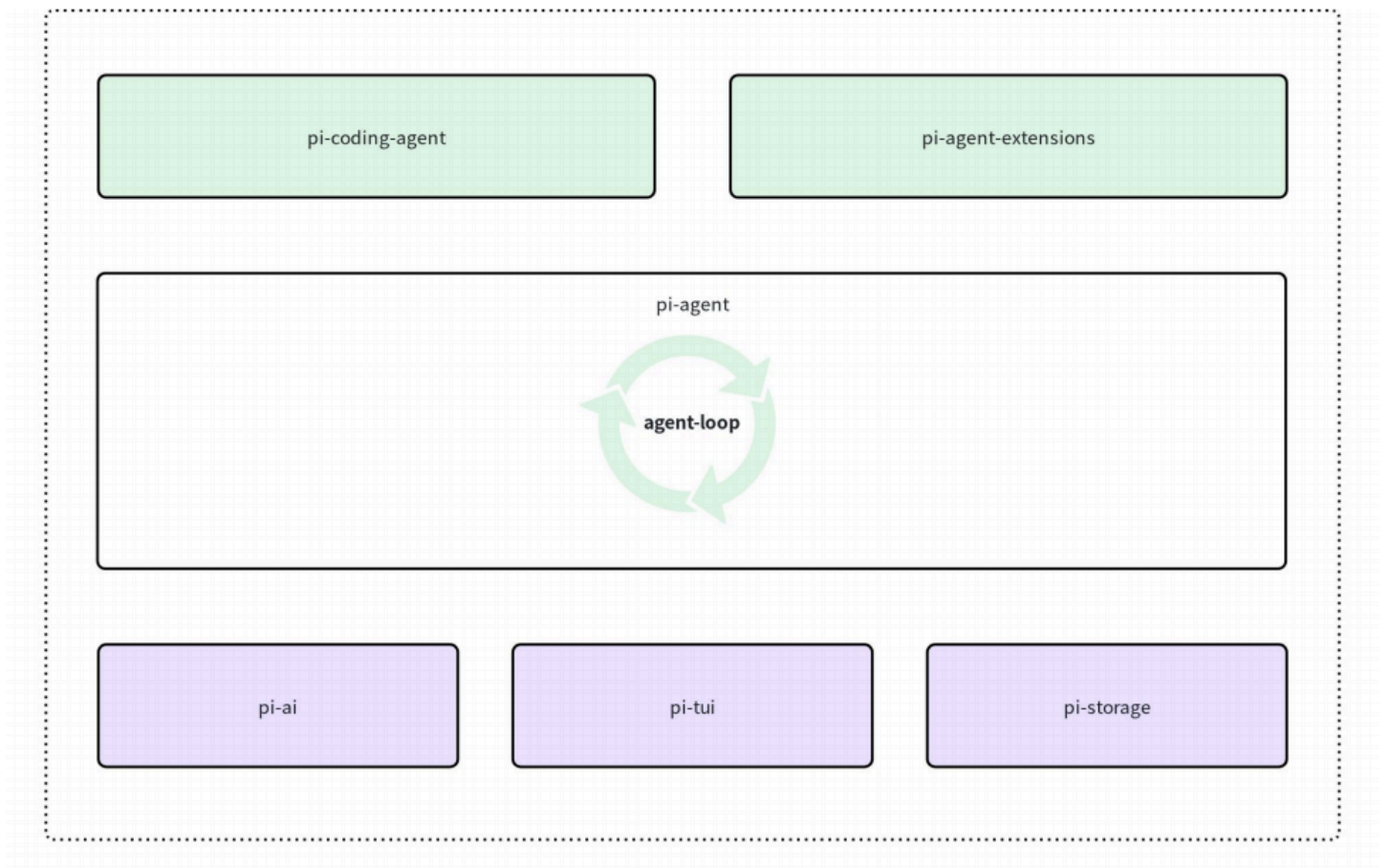
本篇是序篇，会从包结构、一次请求的 body、上下文、Agent Loop 这几个角度，回答一个问题：**Pi 是怎么把一个只能一问一答的大模型，变成一个能读文件、能跑命令、能改代码的 Agent 的？**

先简单交代一下背景。Pi（中文可读作 π ：pai）是一个开源 Coding Agent，TypeScript 写的，在 GitHub 上拿了 10 万 Star。它能接市面上几乎所有主流模型，把 Agent 最核心的 Loop 封装成一个最小可运行的 Harness。它也在很多爆火产品中得到了应用，比如 2026 年初的现象级开源 Agent —— OpenClaw，它在底层也是依赖 Pi 作为自己的 Agent 核心。

它的设计思想可以概括成：**如无必要，勿增实体。**

一、Pi 的包结构

打开 Pi 的仓库，第一反应是包很多。其实按职责看就这六个：



- **pi-ai**: Pi 的一个抽象层。Pi 最初的设计目标就是能够接入所有的模型，但是每个模型厂商的 API 接口都各不相同，这个包可以抹平不同模型厂商之间 API 的差异，对上层（pi-agent）而言暴露一个统一的接口调用
- **pi-tui**: Pi 的终端 UI 工具，属于 pi-coding-agent 的一个依赖，封装了所有的终端交互和 UI 入口，相当于 coding agent 的一个开发者终端入口
- **pi-storage**: Pi 的存储抽象，它存在的目的是为了将 pi-agent 的 session 接入 DB，可以将会话信息进行持久化
- **pi-agent**: Pi 的核心包，它内部就是 Agent Loop 真正运行的地方，在内部实现了 Agent 全生命周期的钩子，并提供了一个最小可用的 Agent Harness
- **pi-coding-agent**: 外部可以直接接入的 coding-agent，本质上是基于 agent 包的 harness 组件集合，包括 tools、context engineering、extensions、sessions 等

💡 **补充**: pi-coding-agent 没有直接提供 sub-agent、background-bash、sandbox 等常规的 harness 组件，因为它的设计哲学是保持最小可用，这些都可以通过 extensions 来进行支持。

- **pi-agent-extensions**: 这部分是开发者可以自由定制模块，是 Pi 专门供开发者进行扩展的组件，比如 sub agent、sandbox、tools 等都可以通过这个 extensions 来进行扩展，它本质上是一个 TS 的函数

- 当然还有其他的包，比如 pi-server、pi-evals，但不在今天的重点讨论范围之内，就不一一介绍了，大家有兴趣可以自己去看一下～

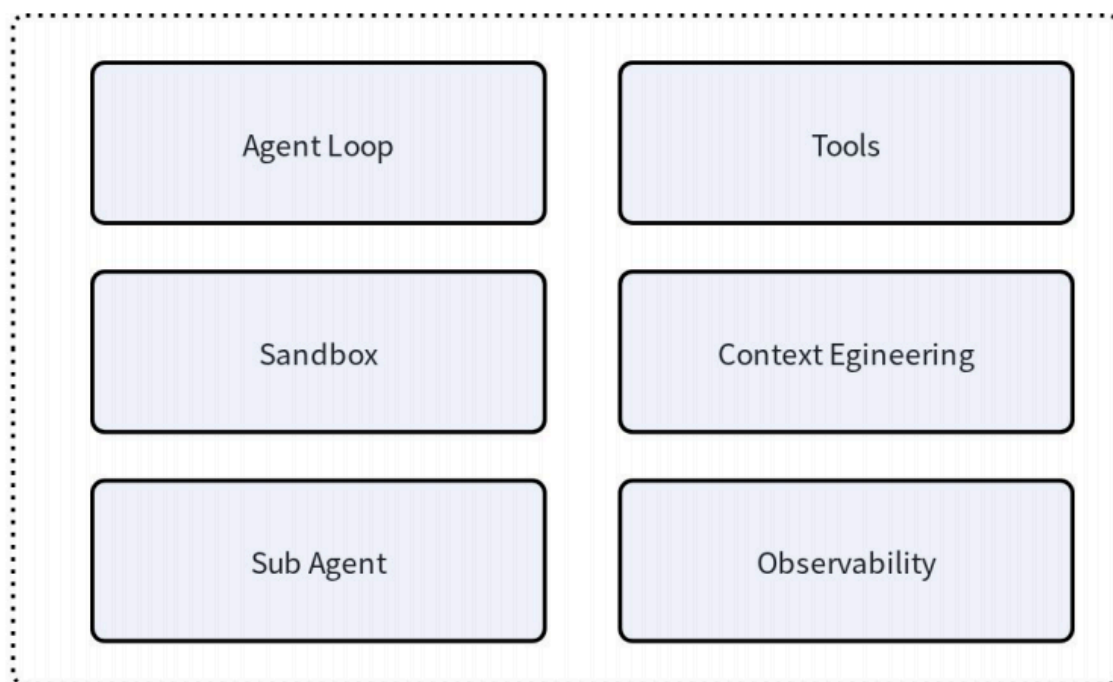
二、Pi 的设计思想：最小可用



看到过一个 Coding Agent 的比较好的定义是：**Coding Agent = Model + Harness**

If you are not the model, you are the harness.

对比一下你就明白它有多克制。一般你以为 Harness 该有的东西——Agent Loop、Tools、Context Engineering、Sandbox、Sub Agent、Observability——Pi 一个都没有：



只提供了 Agent Loop、最基本的 Tools 和简单的 Context Engineering 层。之所以这样，是因为 Pi 的设计目标就是最小可用，官方不希望它的基础包很臃肿；但开发者实际上又需要其他模块，那怎么办呢？可以通过 pi-extensions 来实现，比如：

- 集成 sub agent，可以用 [pi-subagents](#)
- 集成 sandbox，可以用 [pi-agents-tmux](#)



总之需要什么样的扩展，我们就去寻找或者去创造。

保持最小原则的方式就是：

1. 核心保持最小 —— agent-loop
2. 能力交给外围 —— extensions

代价是开箱即用的"全家桶感"弱一点；好处是你读源码的时候，直接就能看到 Agent Loop 的骨架。

三、请求体的结构

谈 Context Engineering 容易停在概念上。我们直接来看：Pi 每次调模型，给模型发的 HTTP body 里到底装了什么？

抓包简化一下，长这样：

代码块

```
1  {
2    "model": "gpt-...",
3    "input": [
4      {
5        "role": "developer",
6        "content": "你是 pi 里的 coding 助手。可用工具: read / bash / edit / write。
当前目录: /Users/.../pi。已加载的 skill: github-triage、incident-analysis... 当前日期: 2026-07-28"
7      },
8      { "role": "user", "content": "请用 github-triage 分析一个 GitHub issue" },
9      { "type": "function_call", "name": "read", "arguments": "
{ \"path\": \"../SKILL.md\" }" },
10     { "type": "function_call_output", "output": "---\\nname: github-
triage\\n..." }
11   ],
12   "tools": [
13     { "name": "read", "description": "...", "parameters": { ... } },
14     { "name": "bash", "description": "...", "parameters": { ... } },
15     { "name": "edit", "description": "...", "parameters": { ... } },
16     { "name": "write", "description": "...", "parameters": { ... } }
17   ],
18   "stream": true
19 }
```

真实的包比这个长几十倍（光 `tools` 里每个工具的 schema 就好几屏），但结构就是两块：

- `input`：上下文。系统提示词 + 用户消息 + 历史上所有的工具调用和结果。
- `tools`：这次允许模型调用哪些函数，每个函数什么参数。

值得一提的是 Pi 的系统提示词特别短。

有一篇文章[The DNA of Coding Agents: Codex CLI vs Gemini CLI vs Claude Code vs Pi — System Prompts Compared](#)专门做过关于不同 Coding Agent 系统提示词的研究：Codex CLI 的 system prompt 大概 6000 token，Gemini CLI 5000–6000，Claude Code 3000–5000，而 **Pi 默认只有 181 个词左右、约 350 token**，这也体现了 Pi 的「小」的理念。



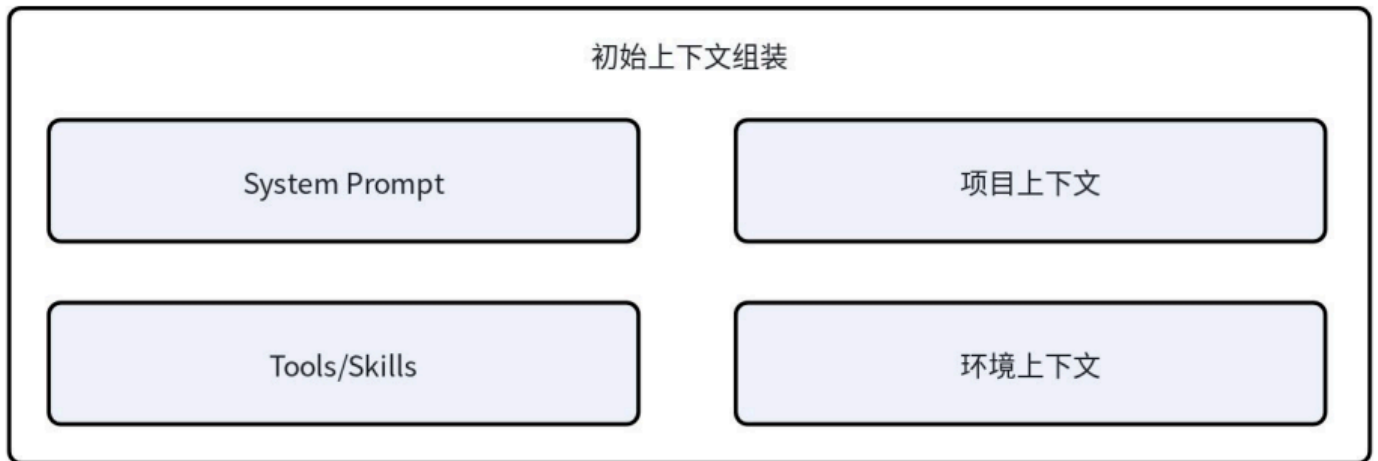
Pi 的系统提示词组装在这里：[pi/packages/coding-agent/src/core/system-prompt.ts](https://github.com/pi/packages/coding-agent/src/core/system-prompt.ts)

四、Context Engineering

知道了发什么，再看怎么管理上下文。

Context 的初始化

每次会话开始，Pi 会拼四块东西进上下文：



1. **System Prompt**: 内置的那一小段，也可以在项目根目录放一个 `.pi/SYSTEM.md` 覆盖；
2. **Tools / Skills 清单**: 工具或技能有变化时重新算一次，注入可用列表；
3. **项目上下文**: 自动从工作目录往上找 `AGENTS.md`（也认 `CLAUDE.md`），把项目约定喂给模型；
4. **环境信息**: 当前 Agent 的工作目录、项目的重点文档路径等辅助信息，帮助大模型正确判断当前工作目录

Context 的压缩

当前主流的所有 Agent 必然都会做上下文的压缩，主要是为了解决历史的 Messages 随着时间推移不断堆积的问题，几乎所有 Coding Agent 的实现思路都差不多。

Pi 的触发条件是：当前上下文 token 数 > 模型窗口 - 预留的 16384 token。压的时候它保留最近 20000 token，把更早的历史交给模型总结成一段 summary，再把 `system prompt + summary + 最近消息` 作为下一轮的输入。

还有一个很贴心的设计：**切分支也会触发压缩**。你在 git 切分支后，Pi 会给模型发一段 branch summary，告诉它工作区文件变了什么，免得模型突然发现文件面目全非、以为自己出错了。

Tools 的截断

Context Engineering 不仅体现在初始化的上下文组装和运行时上下文的压缩，更体现在工具调用的过程中，因为前者其实属于 Coding Agent 的低频行为，而工具调用才是真正的高频行为。这个环节的设计哲学就是：**管理工具的输出，尽可能避免往 Context 塞入过多的内容。**

想象一下，我们每一次工具调用的目的是对外界产生影响，而模型需要获取这个工具调用的结果便于采取下一步的行动。如果模型调用了一个工具运行了一个脚本，这个时候脚本返回了海量的内容，如果不加控制，那么便会顷刻间塞满模型上下文，导致任务的完成能力直线下降；即便有上下文压缩也难以以为继，因为压缩必然伴随很多信息丢失，最终的任务质量也会很差。所以我们需要对每个工具的输出做管理，在 Pi 中这个方式主要是截断：

代码块

```
1  export const DEFAULT_MAX_LINES = 2000;           // 2000 行
2  export const DEFAULT_MAX_BYTES = 50 * 1024;       // 50KB
3  export const GREP_MAX_LINE_LENGTH = 500;         // grep 单行 500 字符
```

Pi 的具体限制：

工具	截断方向	行 / 条目限制	字节限制	特有参数
read	head（保留开头）	2000 行	50KB	offset / limit（分页续读）
bash	tail（保留结尾）	2000 行	50KB	全量输出溢写到临时文件
grep	head	100 条匹配	50KB	单行超 500 字符则截断加 [truncated]
find	head	1000 条结果	50KB	—
ls	head	500 个条目	50KB	—

- Pi 的工具截断建立在共享的 `truncate.ts` 之上，核心是「行数 + 字节数」双限制、谁先触发谁生效（默认 2000 行 / 50KB）
- 各工具按语义定制：read 砍尾（保留文件开头）、bash 砍头（保留命令结尾的错误与结果）、grep / find / ls 各设匹配数、结果数、条目数上限

五、Agent Loop

Pi 的 Agent Loop 放在 pi-agent 这个包下。通常情况下 AgentLoop 是一个无限循环，在 Pi 当中使用两个循环来做 AgentLoop，核心是要满足两种情况下的用户介入。我们先来看看这个 AgentLoop 大概结构：

代码块

```

1  while (true){
2      while ( 是否有pendingMessage ){
3          // 消费pendingMessage
4          if (pendingMessages.length > 0) {
5              for message in pendingMessages:
6                  currentContext.messages.push(message)
7                  newMessages.push(message)
8                  pendingMessages = []
9          }
10         // 请求大模型
11         assistantMessage = await streamAssistantResponse(currentContext,
config)
12
13         // 处理assistantMessage, 执行工具等
14
15         // 捞 steering 消息: 用户在 agent 运行中插队的新消息
16         pendingMessages = await getSteeringMessages()
17     }
18
19     // 内层退出: 说明没有工具要继续, 也没有 steering 消息
20     // agent 本来要停了, 这时再看看有没有 follow-up
21     followUps = await getFollowUpMessages()
22
23     if (followUps.length > 0) {
24         pendingMessages = followUps
25         continue
26     }
27
28     break
29 }

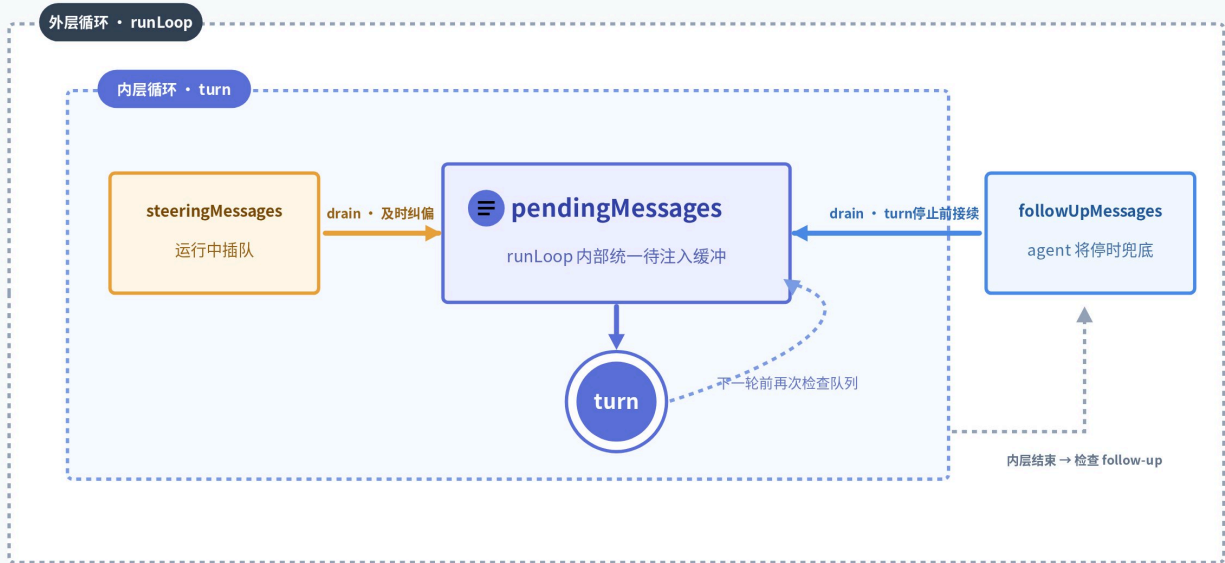
```

- 双循环消费三个队列
- 关注 pendingMessages、followUpsMessages、steeringMessages

Loop 核心流程

AgentLoop 双循环消费队列

两类外部队列，统一汇入下一轮消息缓冲

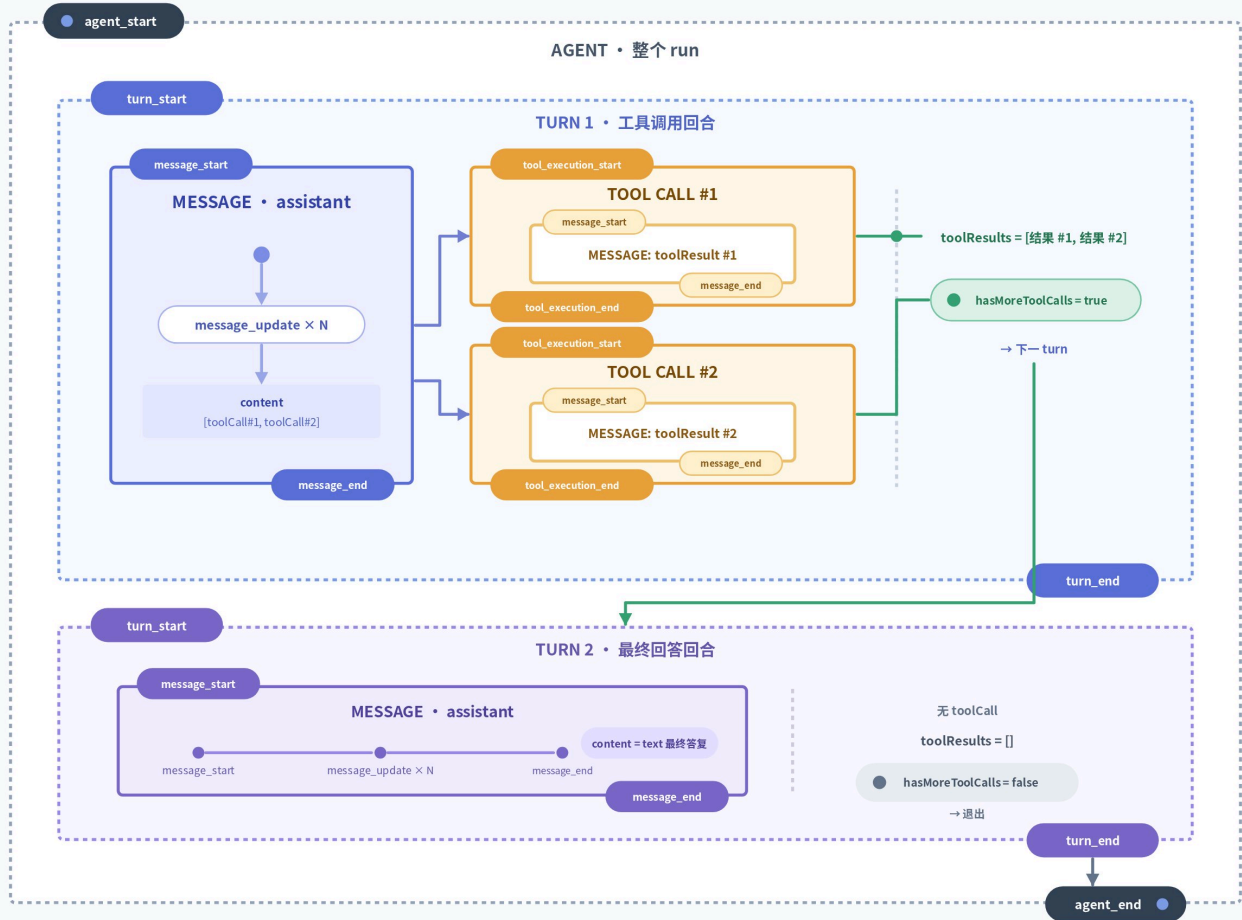


- 在 AgentLoop 双循环中维护了两个队列：steeringMessages、followUpMessages
- 它们最终都会进入 pendingMessages 被内层循环消费
- steeringMessages 会在每一个 turn 进行感知，如果有则直接在内层进行介入
- followUpsMessages 会在每一个 runLoop 被消费，如果存在也会最终放入内层消费

Loop 里面的事件

AgentLoop 事件嵌套生命周期

AGENT 包含 TURN, TURN 包含 MESSAGE 与 TOOL CALL



各个事件的详细说明如下：

概念	精确定义	开始 / 结束事件
Agent	一次完整 agent run，从用户任务开始到彻底收工。 一个 Agent run 可以包含多个 turn。	agent_start / agent_end
Turn	一次 assistant response，加上该 response 触发的所有工具执行和 toolResult。	turn_start / turn_end
Message	写入 transcript / context 的消息单元，例如 user、assistant、toolResult。	message_start / message_update / message_end
Tool execution	工具执行过程本身，不是 message；toolCall 是 assistant message 的内容块，toolResult 才是独立 message。	tool_execution_start / tool_execution_update / tool_execution_end

关于 message 的额外说明：

- 在 Pi 中有这么几类 message: `user`、`assistant`、`tool`
- `user` 和 `toolResult` 的 message lifecycle 几乎是一瞬间同步完成的，基本属于为了提供事件完备而定义
- 而 `assistant` 和 `toolExecution` 的 message lifecycle 是有明显的中间过程的

完整顺序可以理解成：

代码块

```
1  turn_start
2
3      message_start(user)
4      message_end(user)
5
6      message_start(assistant)
7      message_update(assistant) × N
8      message_end(assistant)
9
10     tool_execution_start
11     tool_execution_update*
12     tool_execution_end
13
14     message_start(toolResult)
15     message_end(toolResult)
16
17  turn_end
```

讲到这里，你应该能看出来 Pi 的设计取舍：

- 模型层、Runtime 层、产品层、界面层分开；
- 核心只跑 Loop，不堆功能；
- Context 该截的截、该压的压；
- 事件流完整开放，外面想接什么接什么。

基于此，我开发了[行政Agent系统](#)，它有 Python 后端、MongoDB、权限系统、Web 界面，还有一堆行政业务（制度问答、公文流转、跨部门会签）。整套系统长这样：

学校官方文档处理与问答助手 · 系统架构总览

五层架构 × Loop 自进化 × K8s 多部门弹性伸缩



主链路 (实线): 用户请求 → L1 → L2 → L3 → L4 → L5 数据访问 → 逐层返回答案

反馈环 (橙/紫虚线): Feedback Agent 收集信号 → Loop Engine 分析 → 更新 Skills/Hooks/Rules → 反哺 Agent 行为

设计亮点: Loop 自进化闭环 · 部门级 K8s 弹性 · 四层记忆 · Verifier 强制校验防幻觉

中间蓝色那层 (L2 · Harness 多智能体协同层) 才是 Pi Runtime 真正待的位置——Intent、Rewriter、Retrieval、Answer、Verifier 这些 Agent 都跑在它上面。它外面还有接入层、Loop 自进化层、数据检索层和 K8s 部署层，那些全是我们自己的业务代码，不交给 Pi。

写到简历上就是这样：

项目背景

接手学院与校网信息办已有的校内制度问答原型，原系统采用“文档切块 + 向量检索 + 单 Agent 生成”的简单 RAG 链路，存在多版本制度误引、纯向量检索精确信息不稳、Agent 执行不可控、用户反馈无法反哺策略等问题。针对教务、学工、财务、研究生院等多部门制度文档，参与升级为具备可信 RAG、Agent Harness、分层 Memory、反馈自进化 Loop 的校内行政助手 Agent。

解决方案

- **Agent Harness 编排治理**：自建 Harness 层，将问答链路固定为 Intent → Rewrite → Retrieval → Answer → Verify DAG；pi Agent Runtime 负责意图识别、查询改写、答案生成、结果复核和 bad case 归因，自建治理层负责身份鉴权、部门隔离、事实校验、超时降级、灰度发布和自动回滚，避免 Agent 自由规划导致跳过检索或误答；
- **可信 RAG 数据链路**：将 MongoDB 设计为事实平面，维护制度文档 active / archived 状态、版本链、来源哈希和条款元数据；Milvus 仅负责向量候选召回，命中后必须回 MongoDB 回填完整原始条款并校验生效状态，实现答案引用可核验；
- **混合检索与证据召回优化**：针对制度问答中数字、电话、条款号、时间节点等精确信息难召回的问题，设计 BM25 + Milvus + RRF + Reranker 检索链路；BM25 负责关键词和精确信息召回，向量检索负责语义召回，RRF 融合后重排，并由 Query Rewriter 结合术语表和会话实体生成多组查询；
- **分层 Memory 架构**：将记忆拆分为 Redis 工作记忆、情景记忆、用户语义记忆、组织知识记忆和程序性策略记忆；通过 MemoryContextBuilder 按权限、时效、权威级别和 token 预算裁剪上下文，避免会话摘要、部门 FAQ 或模型推断被误当成官方制度引用；
- **反馈自进化 Loop**：实现 Execute → Observe → Reflect → Adapt → Deploy 闭环，采集点赞、点踩、纠错、追问、复制和 Verifier 结果；通过 Reflect 将 bad case 归因为 retrieval / intent / generation / knowledge_gap，并生成 Skill / Hook / Rule 候选，经历史 Trace 回放、稳定哈希灰度和指标监控后发布或回滚。
- **可执行 Skill Workflow**：将 Skill 从普通提示词升级为可执行策略单元，支持查询词扩展、top-k 调整、回答模板切换、校验约束注入，并记录实验分组、触发次数、成功率和版本效果，用于验证反馈是否真实改变下一轮执行路径。

项目成果

- 在 1,086 份制度文档、28,640 条语义条款、1,200 条分层评测问题上完成规模化验收，Recall@5 达到 93.4%，MRR 0.862，引用正确率 96.1%，部门路由准确率 94.2%；
- 综合回答准确率从基础向量 RAG 的 82.6% 提升到 93.1%，其中混合检索与事实治理贡献 7.2 个百分点，两轮 Skill 灰度迭代贡献 3.3 个百分点。
- 无依据问题安全拒答率达到 97.3%，单部门问答 P95 3.6 秒，跨部门问答 P95 5.4 秒。
- 支持按部门拆分 Agent Service，通过 DEPT_ID 强制过滤和全局 Orchestrator 并行合并；完成 1 → 20 Pod 压测，200 并发下吞吐从 8.7 req/s 提升至 57.9 req/s。（算法岗可以删除这一条）

如果你想找一个已经实现好的、可以直接写在简历上的、基于 pi agent 开发的自进化 multi-agent 项目，可以用我这个。

项目的详细介绍见：<https://mp.weixin.qq.com/s/1qmJ5PiShtf48fhM3LaGow>

扫码加入知识星球即可获取：



本系列后面四篇，我们要做的事就是：取 Pi 它的 `pi-agent-core + pi-ai` 两层，嵌进自己的产品里。

下一篇预告 | 01 看懂 Pi：选对开发层，跑通第一个 Agent

Pi 那一堆包、五个调用入口，到底从哪个下手？我们建一个 `pi-minimal` 目录，装依赖、配模型、写 30 行代码，亲手看到第一句流式回复打在终端上。